

DDR3 Memory Controller

(DDR3MC)

Technical Design Documentation

Source Code Reference & Module Description

Contents

1. Project Overview	4
1.1 Key Characteristics	4
1.2 System Interface Command Set	5
1.3 DDR3 DRAM Operations	5
2. System Architecture	6
2.1 Block Diagram (Textual).....	6
2.2 Operating Phases.....	6
2.3 Clocking & Reset.....	7
2.4 Queues & Data Path	7
2.5 Status Register	7
3. Source File Descriptions	8
ddr3mc_size_parameters.vh	8
ddr3mc_timing_parameters.vh.....	8
ddr3mc_user_data.vh.....	9
ddr3mc_phy_parameters.vh	9
ddr3mc_fpga_wrap.sv	9
ddr3mc_top.v.....	10
ddr3mc_test_driver.sv	10
ddr3mc_flow_control.v	11
ddr3mc_imux.v	11
ddr3mc_queues.v	12
ddr3mc_cfg.v.....	12
ddr3mc_init_engine.v	12
ddr3mc_processing_engine.v	13
ddr3mc_rw_control.v	13
ddr3mc_datapath.v.....	13
ddr3mc_sram.v.....	13
ddr3mc_iodelay_control.v	13
ddr3mc_fifo_sync.v / _piso.v / _sipo.v	14
ddr3mc_cal_engine.v	14
ddr3mc_cal_flow_control.v.....	14
ddr3mc_cal_omux_pipe.v / ddr3mc_cal_imux_pipe.v	15
ddr3mc_cal_preamble_det.v	15
ddr3mc_cal_clock_centering.v.....	15
ddr3mc_cal_dq_detect.v	15
ddr3mc_cal_write_leveling.v	15

ddr3mc_cal_write_dqs_centering.v	16
ddr3mc_cal_read_write_sanity_check.v	16
Calibration wrapper modules	16
ddr3mc_phy_top.v	16
ddr3mc_phy_data_lane.v / ddr3mc_phy_byte_lane.v	17
ddr3mc_phy_dq_bit_lane.v / _dqs_bit_lane.v / _dm_bit_lane.v	17
ddr3mc_phy_control_lanes.v	17
ddr3mc_phy_ck_generator.v	17
ddr3mc_phy_pvt_cal.v + cal.v	17
ddr3mc_sim_bfm.sv	18
ddr3_sim_sodimm_x8_2R.sv	18
ddr3.sv	18
4096Mb / 8192Mb_ddr3_parameters.svh	18
spd_eeprom.sv + 24AA02.v + eeprom.txt	19
Testbenches	19
4. Operational Flows	21
4.1 Power-Up Bring-Up Sequence	21
4.2 DDR3 JEDEC Initialization (init_engine)	21
4.3 Calibration Sequence (cal_engine)	21
4.4 Read / Write Transaction Flow	22
4.5 Refresh Handling	22
5. Module Summary	23
6. DDR5 Migration — Reusable Components	25
6.1 Reusable As-Is (parameter-level changes only)	25
6.2 Reusable Architecture, Significant Extension	26
6.3 DDR5-Specific New Work (low reuse)	27
6.4 Verification & Build Collateral	27
6.5 Net Assessment	28

1. Project Overview

The DDR3 Memory Controller (DDR3MC) is an RTL subsystem that bridges a wide 512-bit synchronous system interface to a standard JEDEC DDR3 SDRAM DIMM. It presents a simple command / address / data port to the rest of the SoC and hides the full complexity of DDR3 operation: SPD-driven configuration, the JEDEC power-up initialization sequence, a complete read- and write-path timing calibration, DRAM bank management with full timing enforcement, periodic refresh, and a custom DDR3 PHY built from foundry hard-macro I/O cells.

On power-up the controller walks through four phases in order — Configuration, Initialization, Calibration, and Run. During Configuration it loads DRAM geometry and timing parameters (from an SPD EEPROM in simulation). During Initialization it drives the JEDEC reset / CKE / Mode-Register / ZQ-calibration sequence on each rank. During Calibration it trains the read and write data eyes (preamble detection, clock centering, per-bit DQ deskew, write leveling, write-DQS centering, and a final read/write sanity check). Only once all three complete does it raise the RW_READY status bit and begin accepting traffic, performing activate / read / write / precharge / refresh operations while honoring every DDR3 timing constraint.

This release is the **ASIC-development variant** (the repository's active branch). In it the Xilinx UltraScale+ PHY primitives have been replaced by foundry hard-macro behavioral models — an integrated DDR I/O cell (IGIDDRT01A), programmable input/output delay lines with their DLLs (DELAY_S1/S4, DLL_TOP_S1/S4, DLL_DCC, the IGADLLT04A DLL cells), and DDR capture/launch registers (IDDRE1 / ODDRE1). A dedicated ZQ / PVT block calibrates the output-driver and termination impedance. In simulation the DDR3 loop is closed against the official Micron DDR3 JEDEC model, arranged as a dual-rank ×8 SO-DIMM, with DIMM geometry supplied over I2C by an SPD EEPROM model.

1.1 Key Characteristics

Parameter	Value	Notes
Controller / PHY clock	200 MHz	sys_clk / clk_0 — DDRMC_CLK_PERIOD_ps = 5000
DDR3 bus signaling	DDR (2 beats / clk)	Double-pumped through PHY IDDR / ODDR registers
System data bus width	512 bits	64 bytes per transaction (SYS_DATA_WIDTH)
System address width	33 bits	Byte address (SYS_ADDR_WIDTH)
DDR3 DQ width	64 bits	DDR3_DQ_WIDTH — eight ×8 byte lanes
DDR3 DQS / DM width	8 each	One differential strobe + one mask per byte lane
DDR3 address / bank width	16 / 3	DDR3_A_WIDTH / DDR3_BA_WIDTH (8 banks)
DIMMs supported	Up to 3 (1 enabled)	CFG_DIMM0/1/2_CONN; DIMM0 connected
Ranks	2 on DIMM0	CFG_DIMM0_NRANKS = 2

Parameter	Value	Notes
Burst length	BL8 (fixed)	Eight-beat bursts, programmed in MR0
Row / Column bits	16 / 11	Configurable from SPD (cfg module)
Config interface	32-bit data / 16-bit addr	SPD-driven (SYS_CFG_DATA/ADDR_WIDTH)
Queue depth	4 entries	CMDQ / WDQ / RDQ (CMDQ_DEPTH)
Default CAS latency (CL)	6 nCK	ptCL_nCK; CWL derived per JEDEC
Sync FIFOs used	5	CMDQ, WDQ (PISO), RDQ (SIPO), RDQ-buffer, plus cfg

1.2 System Interface Command Set

Commands enter the controller on the 4-bit sys_cmd port (defined in ddr3mc_size_parameters.vh). WRITE and READ are issued by the host; REF is generated internally by the flow controller; the remaining opcodes are reserved/defined for future use.

Command	Encoding	Description
WRITE	4'b1001	512-bit (64-byte) write transaction
READ	4'b1010	512-bit (64-byte) read transaction
PD	4'b1011	Power-down (defined)
MRS	4'b1100	Mode Register Set (defined)
CFG	4'b1101	Configuration access (defined)
NOP	4'b0111	No operation
REF	4'b1110	Refresh — generated internally per rank on tREFI

1.3 DDR3 DRAM Operations

Internally the controller translates host commands into the native DDR3 command set, driven onto the RAS_n / CAS_n / WE_n / CS_n pins by the initialization, calibration, and processing engines.

Operation	Issued by	Description
ACT (Activate)	rw_control	Open a row in a bank before column access
READ / WRITE (CAS)	rw_control	Column access, BL8, auto-precharge disabled
PRECHARGE / PRECHARGE-ALL	rw_control	Close one bank's row / all rows (A10 = 1)
REFRESH	rw_control (REF state)	Per-rank auto-refresh paced by the tREFI counter
MRS (MR0–MR3)	init_engine	Program burst length, CL, CWL, drive, ODT, etc.
ZQCL	init_engine	ZQ calibration long, once per rank at init

Operation	Issued by	Description
MPR Read	calibration blocks	Multi-Purpose Register reads for read-side training

2. System Architecture

The DDR3MC is organized as a master flow-control FSM that sequences three operational engines (initialization, calibration, processing) through a shared interface mux into a custom DDR3 PHY. A configuration block supplies all timing and geometry; three queues decouple the host from DRAM timing.

2.1 Block Diagram (Textual)

```

ddr3mc_fpga_wrap                                <- chip / FPGA boundary
|
+-- ddr3mc_clkgen          (MMCM: 200 MHz sys_clk, locked)
+-- ddr3mc_test_driver     (stimulus: SPD config + R/W loops / VIO)
+-- ddr3mc_top             <- functional integration top
    |
    +-- ddr3mc_flow_control (master FSM: CFG->INIT->CAL->RUN)
    +-- ddr3mc_cfg          (SPD -> timing params + geometry)
    +-- ddr3mc_queues       CMDQ | WDQ(PISO) | RDQ(SIPO)
    +-- ddr3mc_init_engine  (JEDEC reset/CKE/MRS/ZQCL per rank)
    +-- ddr3mc_cal_engine   ----- cal_flow_control + o/imux_pipe
        |
        +-- preamble_det    clock_centering  dq_detect
        |
        +-- write_leveling  write_dqs_centering
        |
        +-- read_write_sanity_check
    +-- ddr3mc_processing_engine
        |
        +-- rw_control (rank0)  rw_control (rank1)
        |
        +-- datapath  (WDQ->DQ launch, DQ capture->RDQ)
    +-- ddr3mc_imux        (init/cal/proc command mux)
    +-- ddr3mc_phy_top
        +-- ck_generator    control_lanes  (A/BA/RAS/CAS/WE/CS..)
        +-- data_lane -> byte_lane x8 -> dq/dqs/dm bit_lane
        +-- pvt_cal (cal.v : ZQ driver/ODT impedance)
    |
    +-- ddr3mc_sim_bfm  (SIM)  -> ddr3 x8 2R SO-DIMM model + SPD EEPROM
                                   [SPI/I2C SPD over i2c_scl/sda]

```

2.2 Operating Phases

The flow controller advances through four phases after the MMCM locks and the PHY reports ready (phy_ready). Each phase enables one engine and waits for its done handshake before moving on. Refresh is interleaved into the Run phase.

Phase	Driving module	Description
Configuration	ddr3mc_cfg	Loads timing (tRCD, tCL, tRFC, ...) and DIMM geometry (ranks, row/col/bank bits, address ranges) from the config register interface, sourced from SPD.

Phase	Driving module	Description
Initialization	ddr3mc_init_engine	Runs the JEDEC power-up sequence per rank: assert/deassert RESET_n, raise CKE, program MR2/MR3/MR1/MR0, issue ZQCL.
Calibration	ddr3mc_cal_engine	Six-step read/write data-path training that sets the IDELAY/ODELAY taps and validates the path.
Run	ddr3mc_processing_engine + ddr3mc_flow_control	Pulls commands from CMDQ, performs ACT/READ/WRITE/PRECHARGE with timing checks, and inserts per-rank REFRESH on tREFI.

2.3 Clocking & Reset

Signal	Description
sys_clk / clk_0	200 MHz controller and PHY fabric clock, generated by the ddr3mc_clkgen MMCM from the differential board clock gclk_p/n.
half_clk (PHY)	Half-rate clock toggled inside ddr3mc_phy_top, used by the DDR launch/capture logic.
sys_reset_n / greset_n	Active-low system reset; the flow controller only leaves IDLE once sys_clk_locked and phy_ready are both high.
ddr3_RESET_n	DRAM hardware reset, driven low then high by the init engine during the JEDEC power-up window.

2.4 Queues & Data Path

Three queues (instantiated in ddr3mc_queues) decouple the 512-bit host side from the 128-bit-per-beat DRAM data path. All are single-clock (synchronous) FIFOs, depth 4.

Queue	FIFO type	Description
CMDQ	ddr3mc_fifo_sync	Command + address queue. Width = ADDR(33) + CMD(4) = 37 bits. Written by the host when RW_READY; read by the flow controller.
WDQ	ddr3mc_fifo_sync_piso	Write-data queue, Parallel-In Serial-Out. Accepts a 512-bit word and emits 128 bits per read to the datapath launch logic.
RDQ	ddr3mc_fifo_sync_sipo	Read-data queue, Serial-In Parallel-Out. Accepts 128 bits per captured beat and assembles a 512-bit word for the host.
RDQ buffer	ddr3mc_fifo_sync	Elastic buffer inside the datapath that absorbs read bursts before the RDQ; a room threshold drives rdq_buf_ready (read back-pressure).

2.5 Status Register

The 8-bit sys_status word (maintained by ddr3mc_flow_control) reports bring-up progress and host readiness.

Bit	Name	Meaning
0	INIT_DONE	DDR3 JEDEC initialization complete
1	CFG_DONE	Configuration registers loaded
2	CAL_DONE	Read/write calibration complete
3	RW_READY	Controller ready for host read/write (set when CFG+INIT+CAL done and no queue is full)
4	CFG_READY	Configuration phase active — host may write config registers
5–7	Reserved	Driven to 0

3. Source File Descriptions

This section documents every source file in the DDR3MC project, organized by folder: shared parameter headers (include/), the RTL design (rtl/ — grouped into the controller core, the calibration subsystem, the PHY, and foundry hard macros), the simulation environment (sim/), build scripts (scripts/), and FPGA constraints (const/).

include/ *Shared parameter headers, included by all RTL and simulation files*

ddr3mc_size_parameters.vh

Master sizing header. Defines the rank/DIMM counts, the DDR3 and system interface widths, the command opcodes, the queue parameters, the host-to-DRAM address map, the status-bit positions, and the Mode-Register field widths. Derived parameters such as CFG_NUM_RANKS and CFG_RANK_WIDTH are computed from the user-data defines.

Group	Key parameters
Ranks & DIMMs	CFG_NUM_DIMMS, CFG_NUM_RANKS, CFG_RANK_WIDTH, CFG_BANK_WIDTH=3, CFG_ROW_WIDTH=16, CFG_COL_WIDTH=11
Command opcodes	CMD_OPCODE_WRITE/READ/PD/MRS/CFG/NOP/REF (4-bit)
DDR3 interface	DDR3_A_WIDTH=16, DDR3_BA_WIDTH=3, DDR3_DQ_WIDTH=64, DDR3_DQS_WIDTH=8, DDR3_DM_WIDTH=8
System interface	SYS_DATA_WIDTH=512, SYS_ADDR_WIDTH=33, SYS_CMD_WIDTH=4, SYS_STATUS_WIDTH=8, SYS_CFG_DATA_WIDTH=32, SYS_CFG_ADDR_WIDTH=16
Queues	CMDQ_WIDTH=ADDR+CMD, CMDQ_DEPTH=4; WDQ 512-in/128-out; RDQ 128-in/512-out; bit-field LSB/MSB slicing constants
Address map	AMAP_COL/BANK/ROW MSB:LSB — maps the 33-bit byte address onto column/bank/row
Status bits	STATUS_BIT_INIT_DONE[0], CFG_DONE[1], CAL_DONE[2], RW_READY[3], CFG_READY[4]
Mode Register widths	MR0_CL, MR1_DIC/RTT/AL, MR2_CWL/RTTWR field widths

ddr3mc_timing_parameters.vh

Timing header. Computes every DDR3 timing constraint in clock (nCK) units from the picosecond clock period (DDRMC_CLK_PERIOD_ps), and defines the configuration-register address map used to load timing/geometry values at runtime.

Group	Contents
Timing in nCK	ptRCD, ptCCD, ptCL, ptCWL, ptR2PRE, ptW2PRE, ptWTR, ptRTW, ptRFC, ptRP, ptWR, ptRTP, ptburst, ptXPR, ptMRD, ptMOD, ptZQCL, pt200US, pt500US, ptREFI
Helper	max() integer function for clamping timing minimums
CfgReg addresses	CFGREG_ADDR_* map for control, per-DIMM geometry (connected, ranks, row/col/bank bits, capacity, bus width, module/device type) and timing overrides (tCL, tWR, tRCD, tRRD, tRP, tRAS, tRC, tRFC, tWTR, tRTP, tFAW, tCCD, tZQINIT)
DIMM sizes	DIMM_SIZE_4/8/16GB_IN_BYTES and _IN_QUADWORD unit constants

ddr3mc_user_data.vh

User-tunable defines — the few values a board/configuration owner changes. Sets the timescale (1ns/100ps), the clock period (DDRMC_CLK_PERIOD_ps = 5000, i.e. 200 MHz), additive latency, which DIMM slots are connected and their rank counts, and the electrical settings (output-driver impedance DIC, ODT RTT_Nom, dynamic RTT_WR).

ddr3mc_phy_parameters.vh

PHY parameter header. Holds the (currently commented-out) reference set of UltraScale-style delay attributes — DQ/DQS/DM/CK read & write delay tap values, clock-inversion flags, refclk frequency, and update-mode settings. Retained as documentation of the intended per-pin delay configuration.

rtl/ — Controller Core *Top-level integration and the command/data control path*

ddr3mc_fpga_wrap.sv

Top-level FPGA/chip integration wrapper. Instantiates the clock generator (ddr3mc_clkgen MMCM), the functional core ddr3mc_top, and the ddr3mc_test_driver, and — in simulation — the ddr3mc_sim_bfm DRAM model with its clock and reset generators. Under the SYNTHS define it exposes the real DDR3 device pins; otherwise it drives clocks/reset internally and connects the behavioral DIMM.

Sub-instance	Description
ddr3mc_clkgen	Xilinx clk_wiz MMCM producing the 200 MHz sys_clk and a locked flag from gclk_p/n
ddr3mc_top	Functional core of the controller
ddr3mc_test_driver	Stimulus generator (SPD config + read/write traffic) or hardware VIO probe
ddr3mc_sim_bfm	(sim only) DDR3 SO-DIMM + SPD EEPROM bus-functional model

Compile-time switch — SYNTHS: when defined, the module's ports become real FPGA pins (gclk, greset_n, the full ddr3_* bus) and connect to the board. When undefined (simulation), the clock and reset are generated by internal initial blocks and the DDR3 bus connects to ddr3mc_sim_bfm.

ddr3mc_top.v

Functional integration top. Wires together every controller sub-block and the PHY, and contains the output-path muxing that selects between calibration-driven and processing-driven data. It holds no FSM of its own beyond signal routing and the cal/proc selection assignments.

Port summary (system and DDR3 interfaces):

Port	Dir	Width	Description
sys_clk	in	1	200 MHz controller clock
sys_clk_locked	in	1	MMCM locked indication
sys_reset_n	in	1	Active-low reset
sys_cmd	in	4	Host command (WRITE/READ/...)
sys_address	in	33	Host byte address
sys_write_data	in	512	Host write data
sys_write_data_mask	in	64	Write byte mask (tied 0 at wrap)
sys_read_data	out	512	Read data to host
sys_read_data_valid	out	1	Qualifies read data (= !rdq_empty)
sys_read_data_pull	in	1	Host pop of read-data queue
sys_status	out	8	Status flags (see §2.5)
sys_cfg_wr_en / addr / wr_data	in	1 / 16 / 32	Configuration register write port
ddr3_A / BA	out	16 / 3	DRAM address / bank
ddr3_CK0/1_p/n	out	per-DIMM	Differential DRAM clocks
ddr3_CKE0/1, ODT0/1, CS0/1_n	out	per-DIMM	Clock-enable, on-die-term, chip-select (per rank)
ddr3_RAS_n / CAS_n / WE_n / RESET_n	out	1	DRAM command + reset
ddr3_DM	out	8	Data mask (per byte lane)
ddr3_DQ	inout	64	Bidirectional data
ddr3_DQS_p/n	inout	8	Bidirectional differential strobes
ddr3_EVENT_n	in	1	DIMM thermal/event input

Key internal muxing. DQ/DQS output-enable and the launch data (dq_i1/i2, dqs_i1/i2) are selected by STATUS_BIT_CAL_DONE: before calibration finishes the calibration engine drives them; afterwards the processing engine does. The DQS launch is intentionally cross-wired (dqs_i1↔i2) to support write calibration. The command/address pins come from ddr3mc_imux, which arbitrates between the init, cal, and proc engines.

ddr3mc_test_driver.sv

Verification/bring-up stimulus module. Drives the configuration write sequence and then issues a continuous write→read→verify loop to exercise the controller. In hardware it can be driven by a

Xilinx VIO; in simulation it self-sequences. Its FSM is IDLE → CFG_READY → CFG_WRITE (12 register writes) → RW_READY → WRITE → WAIT1 → READ → NEXT, looping over an incrementing address.

State	Action
CFG_READY	Wait for STATUS_BIT_CFG_READY, then begin loading config
CFG_WRITE	Write the 12 SPD/timing config registers via sys_cfg_*
RW_READY	Wait for STATUS_BIT_RW_READY before issuing traffic
WRITE / WAIT1 / READ	Issue a WRITE, wait, then READ to the same (loop_cnt<<3) address
NEXT	Advance the address and repeat

ddr3mc_flow_control.v

Master sequencing FSM and status generator. Brings the controller through Configuration → Initialization → Calibration, then enters the steady-state Run loop where it pulls commands from CMDQ, decodes the address into rank/bank/row/column, enables the processing engine, and inserts per-rank refresh when a tREFI counter expires. It also owns the entire sys_status word and the cfg_err flag.

State	Description
IDLE	Wait for sys_clk_locked && phy_ready
CFG_EN / CFG_DONE	Enable cfg block; wait for cfg_done; latch CFG_DONE status
INIT_DIMM0_EN / _DONE	Run init engine on DIMM0; wait for init_done
CAL_EN / CAL_DONE	Run calibration engine; wait for cal_done
WAIT_CMD_QUEUE_EMPTY	Steady state: if a rank needs refresh go to REF; else if CMDQ has a command and the read buffer is ready, pull it
CMD_PULL / CMD_PULL_WAIT	Pop CMDQ; decode opcode; route READ/WRITE to RW_EN, anything else to CMD_ERR
RW_EN / RW_DONE	Enable processing engine; wait for proc_done
DIMM0_RANK0/1_REF_EN / _DONE	Issue a REFRESH to rank 0 or 1 via the processing engine
CMD_ERR	Flag an unsupported command and return to the wait state

Address mapping. In CMD_PULL_WAIT the FSM checks the address against the DIMM/rank start–end ranges to pick the rank, then slices the byte address into column/bank/row according to the configured {bank, row, col} bit counts (four supported geometries, e.g. {3,16,11}). An unrecognized geometry sets cfg_err.

RW_READY is combinational: it is asserted only when CFG_DONE, INIT_DONE and CAL_DONE are all set and none of CMDQ/WDQ/RDQ is full — providing natural back-pressure to the host.

ddr3mc_imux.v

Command/address interface multiplexer. Selects which engine drives the shared DDR3 command pins (CS_n, CKE, ODT, RAS_n, CAS_n, WE_n, A, BA, RESET_n) based on the one-hot {init_en,

cal_en, proc_en} select. When no engine is active it drives the bus to its idle/deselected state (CS_n high, RAS_n/CAS_n/WE_n high, CKE high). RESET_n follows the init engine and is otherwise held high.

ddr3mc_queues.v

Queue wrapper. Instantiates the three host/DRAM decoupling FIFOs and exposes their write/read/full/empty ports: the command queue (ddr3mc_fifo_sync), the write-data queue (ddr3mc_fifo_sync_piso, 512→128), and the read-data queue (ddr3mc_fifo_sync_sipo, 128→512). All share the single 200 MHz clock and depth 4.

ddr3mc_cfg.v

Configuration-register block. A small IDLE→EN→DONE FSM that, while enabled, accepts writes on the config interface and latches DRAM timing parameters (with sensible JEDEC defaults) and DIMM geometry. It also computes the Mode-Register field encodings (CL→MR0, CWL→MR2) and derives CWL from CL per the JEDEC table, plus the per-rank start/end address ranges from the decoded DIMM size.

Output group	Contents
Timing (nCK)	tRCD, tCCD, tCL, tCWL, tWTR, tRFC, tRP, tWR, tRTP, tWTP, tburst, tXPR, tZQCL, tMRD, tMOD, t200US, t500US, tREFI
Mode Register fields	mr0_CL (from tCL), mr2_CWL (from tCWL), mr1_DIC/RTT/AL, mr2_RTTWR (from electrical defines)
DIMM geometry	dimm0_connected, dimm0_nranks, row/col/bank addr bits, dimm0_sa/ea, per-rank rank0/1 sa/ea

ddr3mc_init_engine.v

DDR3 JEDEC power-up initialization FSM. For each rank it drives the standard bring-up: hold RESET_n low ($\geq 200 \mu\text{s}$), deassert RESET_n and wait ($\geq 500 \mu\text{s}$), raise CKE and wait tXPR, then program MR2 → MR3 → MR1 → MR0 (each followed by tMRD/tMOD), and finally issue ZQCL (wait tZQCL). It repeats for the second rank, asserting the correct CKE/CS per rank.

State group	Action
ASSERT_RESET → WAIT_200US	Hold ddr3_reset_n low for t200US_nCK
DEASSERT_RESET → WAIT_500US	Release reset; wait t500US_nCK
ASSERT_CKE → CKE_HIGH_NOP(_WAIT) → WAIT_TXPR	Raise CKE for the current rank; wait tXPR
MR2 / MR3 / MR1 / MR0 (+_WAIT)	Drive RAS/CAS/WE low with BA + A encoding the MR contents; wait tMRD/tMOD
ZQCL → ZQCL_WAIT	Issue ZQ-calibration long (A10=1); wait tZQCL
RANK_DONE → NEXT_RANK	Advance to the next rank or finish
DONE	Assert init done; return to IDLE when re-disabled

The exact MR field assignments are encoded directly on ddr3_a in each MR state (burst length fixed at 8, CL/CWL programmable, DLL reset in MR0, write-leveling/TDQS disabled in MR1, dynamic ODT in MR2), making the engine a literal transcription of the JEDEC mode-register tables.

ddr3mc_processing_engine.v

Read/write execution wrapper. Instantiates one ddr3mc_rw_control per rank (rank0, rank1) plus the shared ddr3mc_datapath. It steers the proc_en enable to the rank selected by cmd_rank, OR-combines the two ranks' command outputs, and forwards the write/burst/preamble/postamble strobes to the datapath. proc_done is the OR of the two ranks' done.

ddr3mc_rw_control.v

Per-rank DRAM command engine — the controller's largest FSM. It tracks each of the 8 banks' open/closed state and currently-open row, and on a READ/WRITE/REF command issues the correct sequence of ACT / PRECHARGE / READ / WRITE / REF commands while enforcing every relevant timing parameter through a bank of down-counters. Two cooperating state machines run in parallel: state1 sequences the DRAM commands; state2 drives the burst / preamble / postamble data-phase timing.

FSM	States (abridged)
state1 (command)	IDLE, ST_CHK, ACT(_WAIT), READ(_WAIT), WRITE(_WAIT), PRECHARGE(_WAIT), PRECHARGE_ALL(_WAIT), REF(_WAIT), READ_TO_READ, READ_TO_WRITE, WRITE_TO_WRITE, WRITE_TO_READ, DONE
state2 (data phase)	RPREAMBLE, RBURST, RPOSTAMBLE, WPREAMBLE, WBURST, WPOSTAMBLE, WAIT_1/2/3

Bank management & timing. ST_CHK compares the incoming row against the bank's open row: a hit goes straight to READ/WRITE (gated by tCCD, tWTR, tRTW); a miss precharges first (gated by tRTP, tWTP) then re-activates (tRCD). Per-bank counters track tRFC, tREFI, tRCD, tRP, tWR, tRTP, tWTP, tCCD, tWTR, tRTW and the burst length, each with a matching *_cnt_ok flag. A REF command precharges all banks then issues REFRESH.

ddr3mc_datapath.v

Read/write data-eye launch and capture path. On writes it drives the write DQS pattern (preamble + burst) and feeds 128-bit slices from the WDQ onto dq_i1/dq_i2. On reads it captures dq_q1/dq_q2 from the PHY, packs {q2,q1} into a 128-bit word, and pushes it through an elastic RDQ buffer into the RDQ. The buffer's room threshold produces rdq_buf_ready, which back-pressures the command scheduler so reads are only launched when there is space.

ddr3mc_sram.v

Small parameterized dual-port register-file memory (PTR_W address bits, DATA_W data bits). Synchronous write with asynchronous (combinational) read, reset clears all locations. Used as the storage element inside the FIFO variants.

ddr3mc_iodelay_control.v

Generic delay-line control FSM used by the calibration path to drive the IDELAY/ODELAY primitives. Accepts increment / decrement / load requests, pulses the corresponding ce/inc/load controls, and waits the required settling window (12 cycles for inc/dec, 512 cycles for a value load) before asserting done. The VTC (voltage-temperature compensation) enable is held off in this variant.

ddr3mc_fifo_sync.v / _piso.v / _sipo.v

The synchronous FIFO family used for all queues. ddr3mc_fifo_sync is a standard single-clock FIFO (parameterized width and depth) with full/empty flags and room_avail / data_avail counts. The PISO variant stores wide write words and serializes them into narrower read blocks (e.g. 512→128); the SIPO variant assembles narrow write blocks into wide read words (e.g. 128→512). PISO/SIPO use a two-dimensional word array with separate horizontal (within-line block) and vertical (line) pointers.

Note: ddr3mc_fifo_sync.v is an empty placeholder file in the tree.

rtl/ — Calibration Subsystem *Read/write data-eye training; trains the IDELAY/ODELAY taps*

ddr3mc_cal_engine.v

Calibration top. Instantiates the six calibration step blocks, the calibration sequencer (ddr3mc_cal_flow_control), and a large fabric of omux_pipe / imux_pipe selectors that route the DDR3 command, address, launch-data and delay-control signals from whichever calibration stage is currently active, and route the captured read data (dq_q1/q2, dqs_q1/q2) and delay read-backs into the active stage. The six steps are: (1) preamble detection, (2) clock centering, (3) DQ detection, (4) write leveling, (5) write-DQS centering, (6) read/write sanity check.

Calibration step	Purpose
Preamble detection	Find the DQS read preamble using MPR reads, so the read window is captured at the correct beat
Clock centering	Sweep the DQS input delay and center it in the read data eye (MPR-based)
DQ detection	Per-DQ-bit read deskew — find left/right eye edges and load the center delay value
Write leveling	Align the write DQS to CK by sweeping the DQS output delay until the captured CK transitions
Write-DQS centering	Center the write DQ relative to the write DQS by sweeping the DQ output delay (with pre/post sanity)
Read/write sanity check	Write a known pattern and read it back to confirm the full path after all training

ddr3mc_cal_flow_control.v

Calibration sequencer FSM. Steps linearly through the six calibration stages, asserting each stage's start pulse (a 4-bit shift register that gives the stage a few cycles of start) and advancing on the matching done. The disabled WRITE_PER_BIT_DESKEW and WRITE_CALIBRATION stages remain in the enumeration but are commented out of the active path.

```
IDLE -> PREAMBLE_DETECTION -> CLOCK_CENTERING -> DQ_DETECTION
      -> WRITE_LEVELING -> WRITE_DQS_CENTERING
      -> READ_WRITE_SANITY_CHECK -> CAL_DONE
```

ddr3mc_cal_omux_pipe.v / ddr3mc_cal_imux_pipe.v

Per-signal calibration multiplexers, instantiated dozens of times by the cal engine. omux_pipe selects one of the eight stage inputs (preamble, clock-centering, dq-detect, write-leveling, write-per-bit, write-dqs-centering, write-cal, sanity) onto a single output, based on the sequencer state, registering an INIT value when idle. imux_pipe is the inverse — it fans a captured value out to the per-stage consumers. Both are width-parameterized and pipeline-register the selection.

ddr3mc_cal_preamble_det.v

Preamble-detection step. Enters the DRAM MPR (Multi-Purpose Register) mode, issues reads, and captures the preamble plus four burst beats while sweeping the DQS delay (PIPE_INC / DQS_INC), comparing the captured pattern against the expected predefined pattern until the preamble is correctly located, then exits MPR mode.

State group	Action
PRECHARGE_ALL → MPR_ENTER	Precharge, then program MR3 to enter MPR read mode
READ → PREAMBLE/BURST1–4_CAPTURE	Issue MPR read; sample the preamble and four burst beats
COMPARE → CAL → PIPE_INC / DQS_INC	Compare to the expected pattern; step the pipeline / DQS delay
LOAD_RESET_VALUE → MPR_EXIT	Apply the result; exit MPR mode; DONE (or ERR)

ddr3mc_cal_clock_centering.v

Read DQS centering step (MPR-based). Like preamble detection it captures the preamble and four bursts, but sweeps the DQS input delay across the eye, records the passing edges (ADD_EDGES), computes the midpoint (COMPUTE_CENTERING), and loads the centered DQS input-delay value so the strobe samples DQ in the middle of the data valid window.

ddr3mc_cal_dq_detect.v

Per-bit read deskew. With the DRAM in MPR mode it sweeps each DQ bit's input delay, detecting the left and right edges of that bit's eye (LEFT_EDGE_DETECT / RIGHT_EDGE_DETECT), adding the edges and computing the per-bit centering value (COMPUTE_CENTERING → LOAD_CENTER_VALUE). This removes bit-to-bit skew on the read path.

ddr3mc_cal_write_leveling.v

Write-leveling step. Activates a row, writes a DQS toggling pattern, reads it back, and compares to find the point at which the write DQS aligns to CK, sweeping the DQS output delay and detecting right/left edges, then computing and loading the centered DQS output delay. A built-in SANITY_CHECK confirms the result before DONE; failure routes to ERROR. This compensates for fly-by routing skew between CK and DQS across the DIMM.

State group	Action
ACTIVATE_ALL → WRITE → WRITE_BURST1–4	Open a row; write the leveling pattern (DQS preamble + 4 beats)
WTR_WAIT → READ → READ_BURST1–4	Honor write-to-read; read the pattern back
COMPARE → R/L_EDGE_DETECT → DQS_INC	Compare; sweep DQS output delay; find eye edges
ADD_EDGES → COMPUTE/LOAD_CENTERING → SANITY_CHECK	Compute centered delay; load; verify; DONE/ERROR

ddr3mc_cal_write_dqs_centering.v

Write-DQS (write-data) centering step. Writes four bursts and reads them back, sweeping the DQ output delay up and down to find the left and right edges of the write eye, then computes and loads the centered DQ output-delay value. It runs a PRE_CAL_SANITY_CHECK before and a POST_CAL_SANITY_CHECK after applying the new delays, routing to ERROR if either fails. This centers the write DQ relative to the write DQS so the DRAM captures write data cleanly.

ddr3mc_cal_read_write_sanity_check.v

Final end-to-end check. After all delay values are loaded, it performs a configurable number of write-then-read operations (NUMBER_OF_OPERATIONS = 4) with known data and verifies the read-back, confirming that the fully-calibrated read and write paths are functional before the controller declares CAL_DONE.

Calibration wrapper modules

Each calibration step also has a matching *_wrapper.v module (ddr3mc_cal_preamble_det_wrapper, _clock_centering_wrapper, _dq_detect_wrapper, _write_leveling_wrapper, _write_dqs_centering_wrapper, _read_write_sanity_check_wrapper). The wrapper instantiates the core step FSM together with the ddr3mc_iodelay_control delay-line controllers and handles the handshaking and signal width adaptation between the step logic and the PHY delay primitives. The cal engine instantiates the wrappers, not the bare cores.

rtl/ — DDR3 PHY *Physical layer: clock, command, and data lanes plus impedance calibration*

ddr3mc_phy_top.v

PHY top level. Instantiates the clock generator, the data lanes, the command/address (control) lanes, and the PVT/ZQ calibration block, and produces phy_ready. The PHY also generates an internal half-rate clock. phy_ready asserts only when all DQ, DQS, DM and CK DLLs report lock and the PVT calibration reports no error.

```
phy_ready = &dq_dll_lock & &dqs_dll_lock & &dm_dll_lock
            & &ck_dll_lock & !pvt_cal_err
```

Sub-instance	Description
ddr3mc_phy_ck_generator	Generates the differential DDR3 clocks (CK0/CK1) per DIMM via output DDR registers + ODELAY
ddr3mc_phy_data_lane	All eight byte lanes of the data path (DQ/DQS/DM)
ddr3mc_phy_control_lanes	Command/address output path (A, BA, RAS_n, CAS_n, WE_n, CS_n, CKE, ODT, RESET_n)
ddr3mc_phy_pvt_cal	Process/voltage/temperature (ZQ) impedance calibration

ddr3mc_phy_data_lane.v / ddr3mc_phy_byte_lane.v

The data path is built hierarchically. ddr3mc_phy_data_lane uses a generate loop to instantiate eight ddr3mc_phy_byte_lane blocks (one per $\times 8$ byte). Each byte lane contains eight DQ bit lanes, one DQS bit-lane pair, and one DM bit lane, and concatenates their DLL-lock outputs upward. The data_lane aggregates all 64 dq_dll_lock, 16 dqs_dll_lock and 8 dm_dll_lock bits for phy_ready.

ddr3mc_phy_dq_bit_lane.v / _dqs_bit_lane.v / _dm_bit_lane.v

Single-bit I/O cells, each wrapping the foundry hard macros. A DQ bit lane instantiates an input DDR register (IDDRE1), an input delay line (DELAY_S4 $\approx$ IDELAYE3), a tri-state output buffer (OBUFT) and an output delay line (DELAY_S4 $\approx$ ODELAYE3). The DQS bit lane is similar but uses the narrower DELAY_S1 delays and a differential I/O buffer with DCI (IOBUFDS_DCIEN). The DM bit lane drives the per-byte data mask. The IDDRE1 produces the two captured phases (q1/q2) consumed by the datapath and calibration.

ddr3mc_phy_control_lanes.v

Command/address output lanes. Drives each control pin (A, BA, CKE0/1, ODT0/1, CS0/1_n, RAS_n, CAS_n, WE_n, RESET_n) through an output buffer (with the appropriate launch register/delay), translating the imux'd controller command bus onto the physical DRAM pins.

ddr3mc_phy_ck_generator.v

Differential clock generator. For each connected DIMM it produces the CK_p/CK_n forwarded clocks (and their complementary CK1 pair) using output DDR registers driven by a fixed 1/0 pattern and an output delay line, with a DLL-lock output that feeds phy_ready.

ddr3mc_phy_pvt_cal.v + cal.v

PVT / ZQ impedance calibration. ddr3mc_phy_pvt_cal wraps the cal.v engine, which calibrates the output-driver and on-die-termination impedance against the analog ZQ comparator. cal.v drives the pull-up / pull-down impedance codes (pu/puio, pd/pdio), watches the comparator feedback (pcomp/ncomp), supports a manual-override mode, and raises a give-up/error flag if calibration cannot converge — surfacing as pvt_cal_err in phy_ready. This is distinct from the data-eye training done by ddr3mc_cal_engine: cal.v tunes drive strength and termination, not delay taps.

rtl/ — Foundry Hard-Macro Models *Behavioral models of the ASIC I/O / delay / DLL cells (replace FPGA primitives)*

These files are behavioral models of the foundry I/O and timing cells used in the ASIC-development PHY. They stand in for the equivalent Xilinx UltraScale+ primitives and are compiled with the PHY for simulation.

File	Role
iddre1.v (IDDRE1)	Input DDR register — captures DQ on both clock edges into two parallel phases (q1/q2)
oddre1.v (ODDRE1)	Output DDR register — serializes two parallel phases (i1/i2) onto the DDR output
DELAY_S1.v / DELAY_S4.v	Programmable input/output delay lines ($\approx$ IDELAYE3/ODELAYE3); S1 and S4 are tap-count variants; each exposes a DLL_LOCK
DLL_TOP_S1.v / DLL_TOP_S4.v	DLL wrappers that generate the delay-register codes (slave delays) and lock for the S1/S4 delay lines
DLL_DCC_v003.v	DLL with duty-cycle correction (DCC) — provides corrected clock phases
IGADLLT04A_MASTER/SLAVE/MATCH/PD_v003.v	DLL building blocks: master delay, slave delay, matched delay, and phase detector (lead/lag)
IGIDDRT01A.v	Integrated DDR I/O transceiver hard macro, including the PVDDP_H power/IO cells (the physical DDR pad)

sim/ — Testbenches & Models *Simulation environment: DRAM model, SPD, BFM, and testbenches*

ddr3mc_sim_bfm.sv

Top-level simulation bus-functional model. Instantiates one DDR3 SO-DIMM (ddr3_sim_sodimm_x8_2R) and presents it on the controller's DDR3 bus, closing the read/write loop without real hardware. Also carries the I2C side-band (scl/sda/sa) for the SPD EEPROM.

ddr3_sim_sodimm_x8_2R.sv

Dual-rank $\times 8$ DDR3 SO-DIMM model. Instantiates sixteen ddr3 device models — eight $\times 8$ devices per rank (rank0 on CK0/CS0/ODT0, rank1 on CK1/CS1/ODT1) forming the 64-bit data bus — plus an spd_eeprom for configuration. This mirrors a real registered/unbuffered DIMM topology.

ddr3.sv

The official Micron DDR3 SDRAM JEDEC behavioral model (bus-functional). Models the full DDR3 command set, banks, timing checks, and data path; test mode, duty-cycle correction, TCSR and DLL-off mode are not modeled. Configured by the *_ddr3_parameters.svh headers and the den8192Mb / x8 / sg25 defines.

4096Mb / 8192Mb_ddr3_parameters.svh

Micron density-configuration headers for the ddr3.sv model — they select the 4 Gb or 8 Gb part's row/column/bank organization and timing. The build uses den8192Mb (8 Gb, $\times 8$, speed grade 25).

spd_eeprom.sv + 24AA02.v + eeprom.txt

SPD (Serial Presence Detect) model. spd_eeprom wraps a Microchip 24AA02 2 Kbit I2C serial EEPROM model (24AA02.v) addressed by the DIMM's I2C slave-address pins. Its contents are loaded from eeprom.txt (one byte per line, hex) and represent the JEDEC SPD bytes — DIMM geometry, ranks, capacity, bus width and timing — that the controller reads at configuration time over I2C.

Testbenches

File	Description
ddr3mc_tb_top.sv	Full-system testbench: instantiates ddr3mc_fpga_wrap (controller + clkgen + test driver + DRAM BFM) and runs the complete bring-up and traffic flow
tb_ddr3mc_top.sv	Controller-level testbench: instantiates ddr3mc_top (the DUT) directly with driven clock/reset and stimulus
tb_phy_top.sv	PHY-only unit testbench: exercises ddr3mc_phy_top and the hard-macro models in isolation (compiled by the make run_phy_test flow)

scripts/ — Build & IP *Simulation and Vivado build automation*

File	Purpose
Makefile	VCS compile/run flow. Targets: run_test (full design), run_phy_test (PHY only), clean. Compiles with +define+GUC_NO_PG_PIN +PHY_SIM +x8 +sg25 +den8192Mb and opens waveforms in GTKWave
ddr3mc_envsetup.tcl	Vivado environment setup — helper procs that add the include/, rtl/, const/ and sim/ directories to the project filesets and source the IP/define scripts
ddr3mc_define.tcl	Sets the Verilog defines for synthesis (SYNTHS, ILA_DDR3MC_TOP) and simulation (x8, sg25, den8192Mb, SIM), and the xsim runtime
ddr3mc_ip.tcl	Vivado IP generation — creates the clk_wiz MMCM (ddr3mc_clkgen), the VIO (vio_ddr3mc) and the ILA debug cores (ila_ddr3mc_top, ila_ddr3mc_test_driver)

const/ — FPGA Constraints *Xilinx Design Constraints (XDC) for emulation targets*

XDC constraint files for FPGA implementation on two boards — F1A (fpgaA) and F4A (fpgaBD) — in single-rank (x1) and triple-rank (x3) variants. The io_*.xdc files assign package pins and I/O standards for the DDR3 bus (DIFF_SSTL15 clocks/strobes, SSTL15 single-ended); the wrap_*.xdc files constrain the global clock and reset pins.

File	Purpose
ddr3mc_io_F1A_fpgaA_x3.xdc	DDR3 bus pin/IO-standard assignments — F1A board, triple-rank
ddr3mc_io_F4A_fpgaBD_x1.xdc / _x3.xdc	DDR3 bus pin assignments — F4A board, single- and triple-rank
ddr3mc_wrap_F1A_fpgaA_x3.xdc	Global clock (gclk_p/n, DIFF_SSTL15) and reset (greset_n) constraints — F1A

File	Purpose
ddr3mc_wrap_F4A_fpgaBD_x1.xdc / _x3.xdc	Global clock/reset constraints — F4A board variants

doc/ , README.md , rtl/header.* *Project notes and tooling*

File	Purpose
README.md	One-line project note: this is the ASIC-dev build; add GUC_NO_PG_PIN and PHY_SIM defines (plus the usual) for simulation
doc/doc.txt	Placeholder documentation file (empty)
rtl/header.sh + header.txt	Utility script that strips and replaces the standard company header comment block across all Verilog/SystemVerilog files

4. Operational Flows

4.1 Power-Up Bring-Up Sequence

The flow controller (ddr3mc_flow_control) gates everything on the MMCM lock and the PHY ready signal, then runs the three bring-up phases in order before opening the interface.

1. Reset releases; the MMCM locks (sys_clk_locked) and the PHY DLLs lock and ZQ calibration passes (phy_ready). The flow controller leaves IDLE.
2. CONFIGURATION: the cfg block is enabled. The test driver / host writes the SPD-derived geometry and timing into the config registers; cfg latches them and asserts cfg_done. CFG_DONE status is set.
3. INITIALIZATION: the init engine runs the JEDEC power-up sequence on each rank (reset → CKE → MR2/3/1/0 → ZQCL) and asserts init_done. INIT_DONE status is set.
4. CALIBRATION: the cal engine runs the six training steps and asserts cal_done. CAL_DONE status is set.
5. RUN: with CFG_DONE, INIT_DONE and CAL_DONE all set and no queue full, RW_READY asserts and the controller begins servicing host commands and inserting refresh.

4.2 DDR3 JEDEC Initialization (init_engine)

1. Assert RESET_n low and hold for $\geq 200 \mu\text{s}$ (t_{200US_nCK}); CKE held low.
2. Deassert RESET_n; wait $\geq 500 \mu\text{s}$ (t_{500US_nCK}).
3. Raise CKE for the current rank; issue NOPs and wait tXPR.
4. Program MR2 (CWL, dynamic RTT_WR), then MR3 (MPR off), then MR1 (DLL enable, output-driver impedance, RTT_Nom, AL), then MR0 (BL8, CL, DLL reset, write recovery) — each separated by tMRD / tMOD.
5. Issue ZQCL (ZQ calibration long) and wait tZQCL.
6. Repeat for the next rank; when all ranks are done, assert init_done.

4.3 Calibration Sequence (cal_engine)

The calibration sequencer steps through six stages; each trains a portion of the read or write path and loads delay-tap values into the PHY IDELAY/ODELAY lines via the idelay_control blocks.

1. Preamble detection — locate the DQS read preamble (MPR reads) so the read window is captured on the right beat.
2. Clock centering — sweep and center the DQS input delay in the read data eye.
3. DQ detection — per-bit read deskew: find each DQ bit's eye edges and load its centered input delay.
4. Write leveling — align write DQS to CK by sweeping the DQS output delay until CK transitions are captured.
5. Write-DQS centering — center the write DQ against the write DQS by sweeping the DQ output delay (with pre/post sanity checks).

6. Read/write sanity check — write a known pattern and read it back (×4) to confirm the fully-calibrated path before declaring CAL_DONE.

4.4 Read / Write Transaction Flow

1. The host presents WRITE/READ with an address (and write data) while RW_READY is high. ddr3mc_top pushes {cmd,addr} into CMDQ and, for writes, the 512-bit data into WDQ.
2. The flow controller pops CMDQ, decodes the address into rank/bank/row/column, and enables the processing engine for the selected rank.
3. rw_control checks the target bank: on a row miss it precharges (honoring tRTP/tWTP) and activates (tRCD); on a hit it proceeds directly, honoring tCCD / tWTR / tRTW.
4. It issues the READ or WRITE (BL8). For writes the datapath drives the DQS preamble + burst and streams 128-bit slices from WDQ onto DQ; for reads it captures DQ into the RDQ buffer and assembles 512-bit words into RDQ.
5. proc_done returns to the flow controller, which loops back to service the next command or a pending refresh.
6. The host pops read data from RDQ via sys_read_data_pull while sys_read_data_valid (= !rdq_empty) is high.

4.5 Refresh Handling

Each rank carries a tREFI interval counter inside its rw_control. When a counter expires, the flow controller's steady-state loop prioritizes a REFRESH over new commands: it precharges all banks of that rank and issues an auto-refresh (waiting tRFC) through the processing engine, then resumes normal command service. Refresh is thus transparently interleaved with host traffic on a per-rank basis.

5. Module Summary

Module / File	Location	Function
ddr3mc_size/timing/user_data/phy_parameters.vh	include/	Shared sizing, timing, user, and PHY parameters
ddr3mc_fpga_wrap.sv	rtl/	Chip/FPGA top: MMCM, top, test driver, sim BFM
ddr3mc_top.v	rtl/	Functional integration + cal/proc output muxing
ddr3mc_test_driver.sv	rtl/	Config + read/write stimulus (sim) / VIO (hw)
ddr3mc_flow_control.v	rtl/	Master FSM (CFG→INIT→CAL→RUN), status, refresh, address map
ddr3mc_cfg.v	rtl/	SPD-driven timing parameters, MR encodings, DIMM geometry
ddr3mc_queues.v	rtl/	CMDQ / WDQ(PISO) / RDQ(SIPO) wrapper
ddr3mc_imux.v	rtl/	init/cal/proc command-pin multiplexer
ddr3mc_init_engine.v	rtl/	JEDEC power-up: reset/CKE/MRS/ZQCL per rank
ddr3mc_processing_engine.v	rtl/	Per-rank rw_control ×2 + datapath wrapper
ddr3mc_rw_control.v	rtl/	Per-rank bank FSM: ACT/READ/WRITE/PRE/REF with timing
ddr3mc_datapath.v	rtl/	Write launch (WDQ→DQ) and read capture (DQ→RDQ) + RDQ buffer
ddr3mc_odelay_control.v	rtl/	IDELAY/ODELAY tap inc/dec/load control FSM
ddr3mc_sram.v	rtl/	Dual-port register-file storage for FIFOs
ddr3mc_fifo_sync(_piso/_sipo).v	rtl/	Synchronous FIFO family (standard / 512→128 / 128→512)
ddr3mc_cal_engine.v	rtl/	Calibration top: 6 steps + sequencer + o/imux fabric
ddr3mc_cal_flow_control.v	rtl/	Calibration sequencer FSM
ddr3mc_cal_omux_pipe / imux_pipe.v	rtl/	Per-signal calibration stage multiplexers
ddr3mc_cal_preamble_det.v (+wrapper)	rtl/	Read preamble detection (MPR)
ddr3mc_cal_clock_centering.v (+wrapper)	rtl/	Read DQS centering (MPR)

Module / File	Location	Function
ddr3mc_cal_dq_detect.v (+wrapper)	rtl/	Per-bit read deskew
ddr3mc_cal_write_leveling.v (+wrapper)	rtl/	Write leveling (DQS↔CK)
ddr3mc_cal_write_dqs_centering.v (+wrapper)	rtl/	Write-data eye centering
ddr3mc_cal_read_write_sanity_check.v (+wrapper)	rtl/	End-to-end post-calibration check
ddr3mc_phy_top.v	rtl/	PHY top: ck/data/control lanes + pvt_cal, phy_ready
ddr3mc_phy_data_lane / byte_lane.v	rtl/	8-byte data path hierarchy
ddr3mc_phy_dq/dqs/dm_bit_lane.v	rtl/	Single-bit I/O cells (IDDR/IDELAY/OBUF/ODELAY)
ddr3mc_phy_control_lanes.v	rtl/	Command/address output lanes
ddr3mc_phy_ck_generator.v	rtl/	Differential CK generation
ddr3mc_phy_pvt_cal.v + cal.v	rtl/	ZQ driver/ODT impedance calibration
iddre1 / oddre1.v	rtl/	Input / output DDR registers (hard macro)
DELAY_S1/S4, DLL_TOP_S1/S4, DLL_DCC.v	rtl/	Programmable delay lines + DLLs (hard macro)
IGADLLT04A_* , IGIDDRT01A.v	rtl/	DLL cells and integrated DDR I/O macro
ddr3mc_sim_bfm.sv	sim/	DDR3 SO-DIMM + SPD model wrapper
ddr3_sim_sodimm_x8_2R.sv	sim/	Dual-rank ×8 DIMM (16 device models + SPD)
ddr3.sv + *_ddr3_parameters.svh	sim/	Micron DDR3 JEDEC model + density configs
spd_eeprom.sv / 24AA02.v / eeprom.txt	sim/	I2C SPD EEPROM model + contents
tb_ddr3mc_top / ddr3mc_tb_top / tb_phy_top.sv	sim/	Controller, full-system, and PHY testbenches
Makefile / *_envsetup / *_define / *_ip.tcl	scripts/	VCS + Vivado build and IP generation
ddr3mc_io_*.xdc / ddr3mc_wrap_*.xdc	const/	FPGA pin and clock/reset constraints (F1A, F4A)

6. DDR5 Migration — Reusable Components

This section is a forward-looking architectural assessment, not a description of existing code: it maps the current DDR3MC blocks onto a hypothetical DDR5 port and classifies each as reusable as-is, reusable with significant extension, or new DDR5-specific work. The controller was built with a clean separation between the system interface, the phased bring-up sequencer, the queue subsystem, the calibration framework, and the lane-based PHY — and that separation is what makes much of the design portable. The parts that must change are the ones that encode DDR3-specific protocol facts: a single 64-bit channel, eight flat banks, BL8 bursts, and the DDR3 mode-register / initialization scheme.

What makes DDR5 different. Relative to DDR3, DDR5 splits each DIMM into two independent 32-bit sub-channels (40-bit with ECC); organizes the array as 32 banks in 8 bank groups with separate same-/different-bank-group timing; uses a minimum burst length of 16 (BL16) instead of 8; replaces the DDR3 mode-register-set scheme with addressed MRW / MRR commands over a narrower, partly two-cycle command/address bus; moves to 1.1 V pseudo-open-drain signaling with on-DIMM power management (PMIC) and an SPD hub; and adds on-die ECC, data-bus inversion and write-CRC, receiver decision-feedback equalization (DFE), command/address training, and refresh-management features (same-bank refresh, RFM). These are the changes each block below is measured against.

6.1 Reusable As-Is (parameter-level changes only)

These blocks are protocol-agnostic infrastructure. The architecture and most of the RTL carry over unchanged; the work is re-parameterization (widths, depths, target frequency) rather than redesign.

Component	DDR3MC module(s)	Why it carries over	Adjustment for DDR5
Top-level block hierarchy	<code>ddr3mc_top,</code> <code>ddr3mc_fpga_wrap</code>	The controller-core + PHY + queue + flow-control partitioning is independent of the DRAM generation	Re-parameterize bus widths; instantiate the core per sub-channel
Phased bring-up sequencer	<code>ddr3mc_flow_control</code>	The IDLE→CFG→INIT→CAL→RUN gating, status register, and steady-state command/refresh arbitration is a generic control pattern	Add a CA-training phase; redefine the DDR5 status bits
Queue / buffering subsystem	<code>ddr3mc_queues,</code> <code>ddr3mc_fifo_sync /</code> <code>_piso / _sipo,</code> <code>ddr3mc_sram</code>	CMDQ / WDQ / RDQ are generation-agnostic synchronous FIFOs	Widen for BL16 and split per sub-channel
Clock generation	<code>ddr3mc_clkgen</code>	Differential CK plus derived-domain structure is unchanged	Retune the MMCM for the higher DDR5 data rate
Delay-tap controller	<code>ddr3mc_iodelay_control</code>	IDELAY / ODELAY inc / dec / load management is reusable verbatim	Finer tap resolution / range for higher-speed eyes

Component	DDR3MC module(s)	Why it carries over	Adjustment for DDR5
ZQ / impedance calibration	ddr3mc_phy_pvt_cal, cal.v	The ZQ-driven pull-up / pull-down code-search loop is conceptually unchanged	DDR5 termination (RTT) values and code tables

6.2 Reusable Architecture, Significant Extension

These blocks keep their structure and control philosophy, but must be extended with DDR5 protocol facts — bank groups, BL16, the new command encoding, and additional training steps. The skeleton is reused; the contents grow.

Component	DDR3MC module(s)	What is reused	What must be added / extended
Calibration framework	ddr3mc_cal_engine, ddr3mc_cal_flow_control, omux_pipe / imux_pipe	The sequencer-of-training-steps plus the delay-tap-load routing fabric; preamble, clock-centering, per-bit deskew, write-leveling, DQS-centering and sanity-check steps all have DDR5 analogs	A command/address (CA) training step and receiver DFE adaptation; a larger, reordered step set
Bank-management FSM	ddr3mc_rw_control	Per-bank open-row tracking and the timing-counter enforcement engine (tRCD / tRP / tWR / tRTP / tCCD / tWTR ...)	Bank groups (32 banks / 8 BG); same- vs different-BG timing (tCCD_L/S, tRRD_L/S, tWTR_L/S); BL16 burst counters; DDR5 command timing
Write / read datapath	ddr3mc_datapath	The write-launch, read-capture and serialize / deserialize structure	BL8→BL16 slice counts; data-bus inversion and write-CRC; the on-die-ECC boundary
Configuration / param load	ddr3mc_cfg	The SPD-driven parameter-load FSM skeleton	DDR5 timing set + bank-group params; new SPD hub format; the MR model is different (see 6.3)
Command muxing	ddr3mc_imux, imux_pipe	Muxing the command / address pins among the init, cal and proc sources	DDR5 CA encoding (narrower bus, two-cycle commands)
Address mapping	decode in ddr3mc_flow_control	Geometry-driven slicing of the byte address into rank / bank / row / column	Bank-group bits; the split across two sub-channels

Component	DDR3MC module(s)	What is reused	What must be added / extended
PHY lane hierarchy	ddr3mc_phy_top, data / byte / bit lanes, control_lanes, ck_generator	The lane-based decomposition and the per-bit delay structure	Higher-speed I/O primitives and DFE; 1.1 V pseudo-open-drain signaling

6.3 DDR5-Specific New Work (low reuse)

These items have little DDR3 counterpart and represent the bulk of the genuinely new engineering. The first — moving from one 64-bit channel to two independent sub-channels — is the single largest structural change, because the current core is written as a single channel.

Item	Why it is new for DDR5	Primary impact
Dual 32-bit sub-channels	DDR5 splits each DIMM into two independent sub-channels; DDR3MC is one 64-bit channel	Duplicate or parameterize the datapath, command path and flow control per sub-channel — the largest restructuring
Mode-register model	Addressed MRW / MRR commands replace the DDR3 BA-encoded MRS scheme	Rewrite init_engine MR programming and the cfg MR encoding
Initialization sequence	DDR5 power-up differs and interleaves CA training and extra steps	New init-FSM contents (the FSM shape is still reusable)
CA-bus training	The command / address bus is trained at speed in DDR5	A new calibration step plus PHY support
DBI / write-CRC / on-die ECC	Introduced with DDR4 and retained / extended in DDR5 for signal integrity and reliability	New datapath logic and control
Decision-feedback equalization	Required at DDR5 receive data rates	New PHY receive path plus adaptation in the calibration loop
On-DIMM PMIC / RDIMM RCD	DDR5 moves voltage regulation on-DIMM and RDIMMs use a new registering clock driver	Board and PHY / control; SPD-hub access for the PMIC
Refresh management	DDR5 adds same-bank refresh (REFsb), fine-granularity refresh, and RFM for row-hammer	Extend the refresh logic (reuses the per-rank counter pattern)

6.4 Verification & Build Collateral

The verification methodology and build flow are reusable as scaffolding even though their DDR3-specific contents are replaced.

Area	What is reused	What is replaced / updated
Testbench structure	The three-tier testbench split (PHY-only, controller, full-system) and the BFM + SPD-model pattern	Swap the Micron DDR3 device model for a DDR5 model; swap the I2C SPD EEPROM for a DDR5 SPD hub;

Area	What is reused	What is replaced / updated
		rewrite the parameter headers for DDR5 geometries
Build flow	The Makefile / TCL structure (envsetup, define, ip) and the compile-define methodology	Update +defines, file lists, and IP generation (clock-wizard retune for DDR5 rates)
FPGA constraints	The constraint methodology (pin, IO-standard, clock / reset XDC split per board)	Full pinout rework for the DDR5 pinout, 1.1 V signaling, and the two sub-channels

6.5 Net Assessment

Most of the system-level scaffolding is reusable: the block hierarchy, the phased bring-up sequencer and status model, the queue subsystem, the calibration framework and its delay-tap-load fabric, the lane-based PHY decomposition, and the delay / ZQ control. The heaviest new work concentrates in three places — restructuring the single 64-bit channel into two independent DDR5 sub-channels, building the DDR5 protocol layer (the MRW / MRR mode-register model, the revised initialization sequence, and CA-bus training), and adding the DDR5 signal-integrity and reliability features (DBI, write-CRC, on-die ECC, and receiver DFE). In short, the controller's control and dataflow architecture ports well; what changes is the DDR3-specific protocol and electrical layer it was originally specialized for.